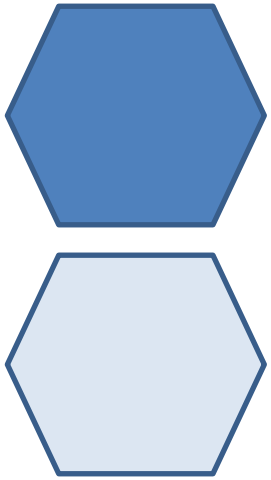
Bioinformatics

Ing. Giulia Fiscon, PhD

fiscon@diag.uniroma1.it

Department of Computer, Control, and Management Engineering
Sapienza University of Rome

- R programming
  Part II: input/output, functions, scripts



Input/Output

Writing Data

There are a few principal functions writing data into R.

- write.table() is used to write data frames into formatted text files. A variable separator can be specified.
- write.csv() is used to write data frames into comma separated variable files.
- write.csv2() is used to write data frames into semicolon separated variable files.
- save, is used to save datasets into a binary file. Data are stored in binary format (more compact!!).

Write.table

- write.table() is used to write data frames into formatted text files,

write.table(x, file, col.names=TRUE, row.names=TRUE, sep=" ", dec=".", quote = FALSE, ...)

x: the object to be written;

file: the name of the file in which the data are stored;

col.names: if TRUE column names are stored;

row.names: if TRUE row names are stored;

sep: the field separator character (default = space);

dec: the character used for decimal points;

quote: if FALSE the string character will not show “”

... : optional arguments;

> write.table(D, "./df.txt", col.names = TRUE, row.names = FALSE, sep = "\t ")

Write.csv

- write.csv() is used to write data frames into formatted text files,

write.csv(x, file, col.names=TRUE, row.names=TRUE, sep=",", dec=".", quote = FALSE, ...)

x: the object to be written;

file: the name of the file in which the data are stored;

col.names: if TRUE column names are stored;

row.names: if TRUE row names are stored;

sep: the field separator character;

dec: the character used for decimal points;

quote: if FALSE the string character will not show “”

... : optional arguments;

> write.csv(b, "./example.txt", col.names = TRUE, row.names = FALSE, sep = ",")

Write.csv2

- write.csv2() is used to write data frames into formatted text files,

write.csv2(x, file, col.names=TRUE, row.names=TRUE, sep=";", dec=",", quote = FALSE, ...)

x: the object to be written;

file: the name of the file in which the data are stored;

col.names: if TRUE column names are stored;

row.names: if TRUE row names are stored;

sep: the field separator character;

dec: the character used for decimal points;

quote: if FALSE the string character will not show “”

... : optional arguments;

> write.csv2(b, "./example.txt", col.names = TRUE, row.names = FALSE, sep = ";")

Save

- save() writes an external representation of R objects to the specified file,

save(...,file, ...)

... : a list of objects to be saved;
file : the name of the file in which the data are stored;
... : optional arguments;

> save(a,b, file = "./example.Rdata")

Reading Data

There are a few principal functions reading data into R.

- read.table() is used to read data frames from formatted text files. A variable separator can be specified.
- read.csv() is used to read data frames from comma separated variable files.
- read.csv2() is used to read data frames from semicolon separated variable files.
- load() is used to reload datasets written with the function save(). Data are stored in binary format (more compact!!).

Reading Data Files with read.table

The read.table function is one of the most commonly used functions for reading data. It has a few important arguments:

- file, the name of a file, or a connection
- header, logical indicating if the file has a header line
- sep, a string indicating how the columns are separated
- colClasses, a character vector indicating the class of each column in the dataset
- nrows, the number of rows in the dataset
- comment.char, a character string indicating the comment character
- skip, the number of lines to skip from the beginning
- stringsAsFactors, should character variables be coded as factors?
- check.names: adjust names of strings? (eg. - is replaced with .)
- quote: specify quoting character

Reading Data Files with read.table

- read.table() reads a file in table format and creates a data frame from it,

  read.table(file, header=FALSE, sep= " ", check.names = F, quote = "", …)

file: the name of the file in which the data are stored;

header: a logical value indicating whether the file contains the names of the variables as its first line;

sep: the field separator character;

check.names: if = FALSE, prevent to change names of strings (otherwise - is replaced with .)

quote: specify quoting character (in this case, no quoting)

... : optional arguments;

> a <- read.table("./example.txt", header = FALSE, sep = ' ')
> b <- read.table("./df.txt", header = TRUE, sep = '\t')

Reading Data Files with read.table

For small to moderately sized datasets, you can usually call read.table without specifying any other arguments

```
data <- read.table("example.txt")
```

R will automatically

- skip lines that begin with a #
- figure out how many rows there are (and how much memory needs to be allocated)
- figure what type of variable is in each column of the table Telling R all these things directly makes R run faster and more efficiently.

```
interactome <- read.table("files/hippie_current.txt",header = F, sep = "\t", check.names = F, quote = "")


matrix <- read.table("files/matrix/matrix_brca.txt",header = T, sep = "\t",check.names = F, quote = "", row.names = 1)
```

Reading Data Files with read.table: large dataset

With much larger datasets, doing the following things will make your life easier and will prevent R from choking.

- Read the help page for read.table, which contains many hints
- Make a rough calculation of the memory required to store your dataset. If the dataset is larger than the amount of RAM on your computer, you can probably stop right here.
- Set comment.char = "" if there are no commented lines in your file

- Use the colClasses argument. Specifying this option instead of using the default can make 'read.table' run MUCH faster, often twice as fast. In order to use this option, you have to know the class of each column in your data frame. If all of the columns are "numeric", for example, then you can just set colClasses = "numeric". A quick an dirty way to figure out the classes of each column is the following

```
initial <- read.table("datatable.txt", nrows = 100)
classes <- sapply(initial, class)
tabAll <- read.table("datatable.txt",
colClasses = classes)
```

- Set nrows. This doesn't make R run faster but it helps with memory usage

Reading Data Files with read.csv

- read.csv() reads a file in table format and creates a data frame from it,

read.csv(file, header=FALSE, sep= ",", …)

file: the name of the file in which the data are stored;

header: a logical value indicating whether the file contains the names of the variables as its first line;

sep: the field separator character;

... : optional arguments;

> a = read.csv("./example.txt", header = FALSE, sep = ',')

Reading Data Files with read.csv2

- read.csv2() reads a file in table format and creates a data frame from it,

read.csv2(file, header=FALSE, sep= ";", …)

file: the name of the file in which the data are stored;

header: a logical value indicating whether the file contains the names of the variables as its first line;

sep: the field separator character;

... : optional arguments;

> a = read.csv2("./example.txt", header = FALSE, sep = ';')

Load

- load() reload datasets written with the function save().

load(file, ...)

File: the name of the file in which the data are stored;
verbose = FALSE: if TRUE item names are printed;
... : optional arguments;

```
> load("./example.RData")
> load("./example.RData", verbose = T)
Loading objects :
m
```

Save and Load the workspace

- In R the workspace can be saved and loaded using:

save.image(file =".RData")
load(file =".RData")

```
> save.image(file = "OutputWorkspace.RData")

> load(file = "OutputWorkspace.RData")
```

Install a package in R

- In R, a package can be downloaded and installed from CRAN-like repositories or from local files;

install.packages(pkgs,rep=getOption("repos"))

pkgs : character vector of the names of packages to be downloaded;
rep : base URL(s) of the repositories to use.
Default CRAN repository.
... : optional arguments;

> install.packages(“igraph”)
> install.packages(path_to_file, repos = NULL, type = ”source”)

Load a package in R

- In R a package must be loaded before being used;

library(package,....)
package : name of the package to be loaded;
... : optional arguments;

> library(datasets) **load a pre-installed datasets available on R**
> library(igraph) **load a specific package installed by the user**
> library() **see all packages installed**

Exercises on input/output

From the Exercise on data.frames, you have created the data.frame D:

- Create a data frame called *D* with the following data:

Firstname	Lastname	Age	Gender	Points
Leonard	Hofstadter	32	M	278
Sheldon	Cooper	34	M	242
Penny	Hofstadter	23	F	312
Howard	Wolowitz	27	M	740
Rajesh	Koothrappali	26	M	177
Bernadette	Rostenkowski	29	F	195

```
> Firstname <- c("Leonard","Sheldon","Penny","Howard","Rajesh","Bernadette")
> Lastname <- c("Hofstadter","Cooper","Hofstadter","Wolowitz","Koothrappali",
"Rostenkowski")
> Age <- c(32, 34, 23, 27, 26, 29)
> Gender <- c("M","M","F","M","M","F")
> Points <- c(278, 242, 312, 740, 177, 195)
> D = data.frame(Firstname, Lastname, Age, Gender, Points)
```

vector names are used as column names

Exercises on input/output

- Save in the textual file “df.txt" the data frame D, using “tab" as variable separator
- Read the data frame stored in the textual file “df.txt“
- Save in the textual file "example.csv" the data frame using ";" as variable separator
- Load the data frame stored in the textual file "example.csv“

Exercises on input/output

- Save in the textual file “df.txt" the data frame D, using “tab" as variable separator

```
> write.table(D, file = "./df.txt", sep= "\t", colnames = T, rownames = F, quote = F)
```

- Read the data frame stored in the textual file “df.txt“

```
> D = read.table(file = "./df.txt", sep = "\t", header = T)
```

- Save in the textual file "example.csv" the data frame using ";" as variable separator

```
> write.table(D, file = "./example.csv", sep = ";", colnames = T, rownames = F, quote = F)
```

- Load the data frame stored in the textual file "example.csv“

```
> K = read.table(file = "./example.csv", sep = ";")
```

Exercises on input/output

- Import the interactome file downloaded from HIPPIE (http://cbdm-01.zdv.uni-mainz.de/~mschaefer/hippie/download.php)
- Mantain only the Entrez Gene ID (numeric gene identifiers) and Score columns
- Rename columns as «source», «target» and «score»
- Filter out interactions with a score < 0
- Write the new data.frame in a tab separated file

Exercises on input/output

- Import the interactome file downloaded from HIPPIE (http://cbdm-01.zdv.uni-mainz.de/~mschaefer/hippie/download.php)
- Mantain only the Entrez Gene ID (numeric gene identifiers) and Score columns
- Rename columns as «source», «target» and «score»
- Filter out interactions with a score < 0
- Write the new data.frame in a tab separated file

```
> interactome <- read.table("files/hippie_current.txt",header = F, sep
= "\t", check.names = F, quote = "")


> interactome <- interactome[,c(2,4,5)]
> colnames(interactome) <-  c("source","target","score")
> interactome <- unique(interactome[interactome$score > 0,])


> write.table(interactome, "files/interactome.txt", col.names = T,
row.names = F, quote = F, sep ="\t")
```

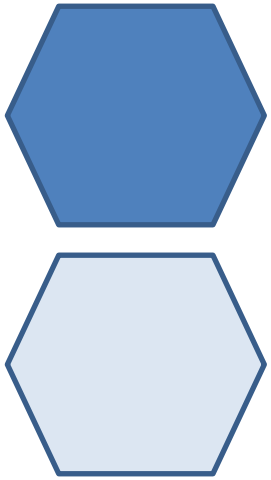
Exercises on input/output

- Create a matrix with 1,000,000 elements and save it using "write.table" and "save"

Exercises on input/output

- Create a matrix with 1,000,000 elements and save it using "write.table" and "save"

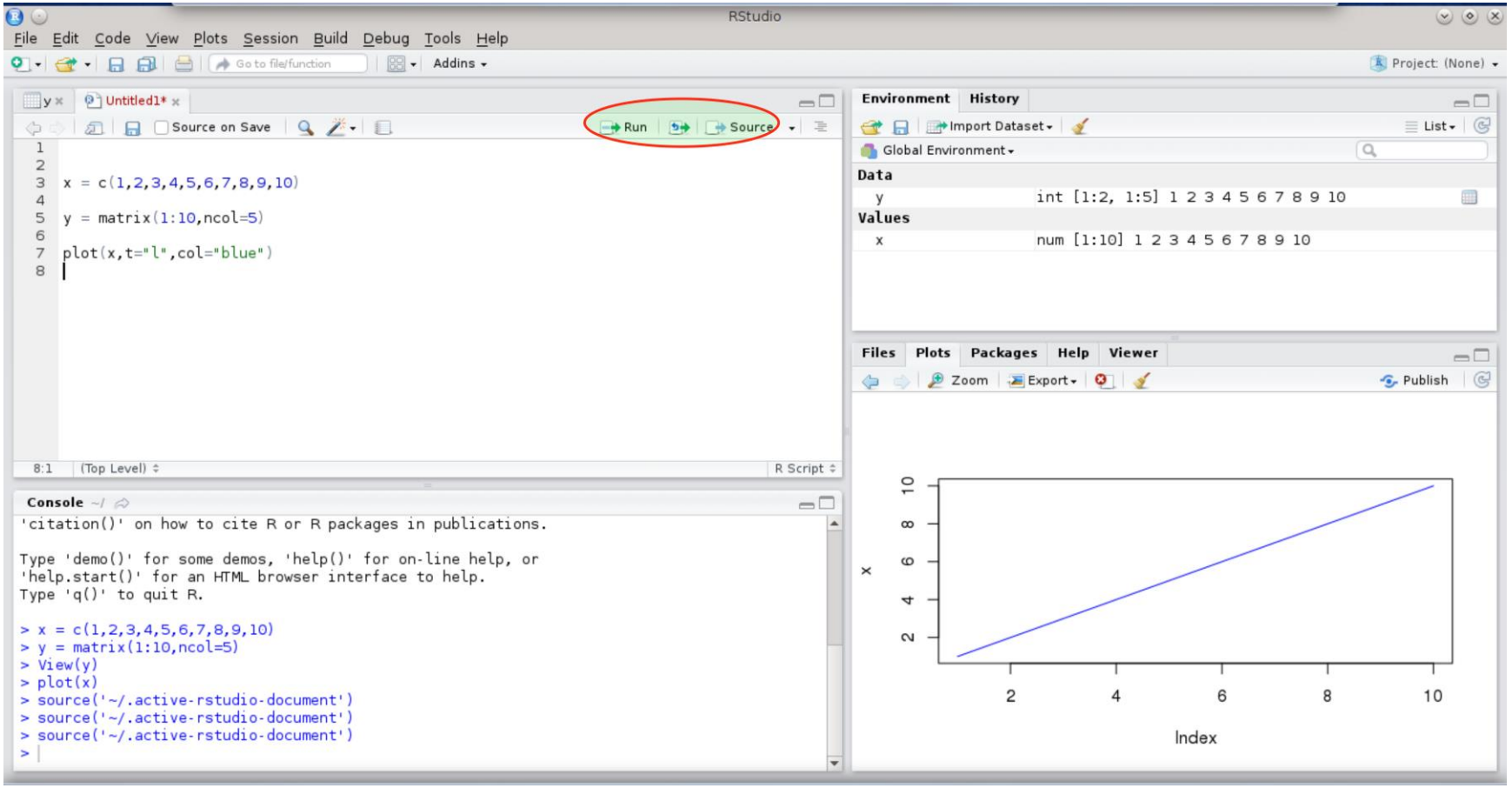
```
> m = matrix(1:1000000, ncol = 100000)
> write.table(m, file = "./example.csv")
> save(m, file = "./example.RData")
```

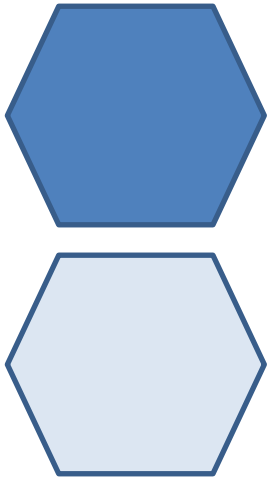


Script

Importing R scripts

- An R-script is simply a text file containing commands;
- It must be in the Working Directory;
- To run each line at time click “run”
- To execute the whole script use source("scriptFile")





Functions

Functions

- Functions are created using the function() directive and are stored as R objects: they are R objects of class “function”.

```
f <- function(<arguments>) {
## Do something interesting
}
```

- Functions can be passed as arguments to other functions
- Functions can be nested, so that you can define a function inside of another function
- The return value of a function is the last expression in the function body to be evaluated.

Function arguments

- Functions have *named arguments* which potentially have *default values*.
- The *formal arguments* are the arguments included in the function definition
- Not every function call in R makes use of all the formal arguments
- Function arguments can be *missing* or might have default values

Arguments matching

- Most of the time, **named arguments are useful on the command line when you have a long argument list** and you want to use the defaults for everything except for an argument near the end of the list
- Named arguments also **help if you can remember the name of the argument and not its position** on the argument list (e.g., plotting).
- Function arguments can also be ***partially* matched**, which is useful for interactive work. The order of operations when given an argument is

    1. Check for *exact* match for a named argument
    2. Check for a *partial* match
    3. Check for a *positional* match

Arguments matching

- R functions arguments can be matched positionally or by name. So the following calls to `mean` are all equivalent

```
> mydata <- rnorm(100)
> mean(mydata)
> mean(x = mydata)
> mean(x = mydata, na.rm = FALSE)
> mean(na.rm = FALSE, x = mydata)
> mean(na.rm = FALSE, mydata)
```

- Although it's legal, I don't recommend messing around with the order of the arguments too much, since it can lead to some confusion.

Defining a function

```
f <- function(a, b = 1, c = 2, d = NULL){

}
```

- In addition to not specifying a default value, you can also set an argument value to NULL

Defining a function

- Arguments to functions are evaluated *lazily*, so they are evaluated only as needed.
- This function never actually uses the argument b, so calling f(2) will not produce an error because the 2 gets positionally matched to a.

```
f <- function(a, b) {
a^2
}

f(2)
## [1] 4
```

```
f <- function(a, b) {
print(a)
print(b)
}
f(45)
## [1] 45
## Error: argument "b" is missing, with no
default
```

Notice that “45” got printed first before the error was triggered. This is because b did not have to be evaluated until after print(a). Once the function tried to evaluate print(b) it had to throw an error.

the "..." Argument

- The **...** argument indicate a variable number of arguments that are usually passed on to other functions. It is often used:

❑ when extending another function and you don't want to copy the entire argument list of the original function

```
myplot <- function(x, y, type = "l", ...) {
plot(x, y, type = type, ...)
}
```

❑ when the number of arguments passed to the function cannot be known in advance.

```
> args(paste)
function (..., sep = " ", collapse = NULL)
> args(cat)
function (..., file = "", sep = " ", fill = FALSE,
labels = NULL, append = FALSE)
```

❑ Generic functions use ... so that extra arguments can be passed to methods

```
> mean
function (x, ...)

UseMethod("mean")
```

Example of customized functions

```
add2 <- function(x,y){
    x + y
}

above10 <- function(x){
    use <- x > 10
    x[use]
}

above <- function(x,n){
    use <- x > n
    x[use]
}

Add_and_multiple2 <- function(x,y){
     s <- x + y
     p <- x*y
     res <- list(sum = s, product = p)
}
```

Example of customized functions

```
#  default value set to 10
above <- function(x,n=10){
    use <- x > n
    x[use]
}

columnmean <- function(y){
    nc <- ncol(y)
    means <- numeric(nc)
    for(i in 1:nc){
        means[i] <- mean(y[,i])
    }

    return(means)
}
# example columnmean(airquality)with NA in the columns
columnmean <- function(y,removeNA = TRUE){
    nc <- ncol(y)
    means <- numeric(nc)
    for(i in 1:nc){
        means[i] <- mean(y[,i], na.rm = removeNA)
    }
    means
}
```

Exercise Script and functions

- Write a script that builds a data.frame with 100 rows, including as columns:
    - Numeric Vector with random values from 150 to 180, named height
    - Numeric Vector with random values from 50 to 180, named weight
    - Numeric Vector with BMI computed as weight/ height squared, when using a customized function to compute the power of height
- As row.names set a character vector of random patients PZ, named PZ_X (e.g., PZ1, PZ2, …)
- Write the data.frame in a txt file tab separeted

Exercise Script and functions

```
rm(list = ls())

options(stringsAsFactors = F)

################################
source("myFirst-functions.R")
################################

pz_list <- paste("PZ",1:100, sep = "")
age <- sample(50:80,100,replace=T)
height <- sample(160:180,100,replace=T)
weight <- sample(50:180,100,replace=T)

h_squared <- power(height)

bmi <- weight/h_squared

df <- data.frame(age = age,
                 height = height,
                 weight = weight,
                 bmi = bmi,
                 row.names = pz_list)

write.table(df, "my_first_dataset.txt", sep = "\t", quote = F,
            col.names = NA, row.names = T)
```

Exercise Script and functions

```
power <- function(x){
  x^2
}

add2 <- function(x,y){
    x + y
}

above10 <- function(x){
    use <- x > 10
    x[use]
}

above <- function(x,n){
    use <- x > n
    x[use]
}
```

MyFirst-Functions.R